Carbon OCR

The fast OCR engine in C++ and CUDA

Measured, Not Claimed

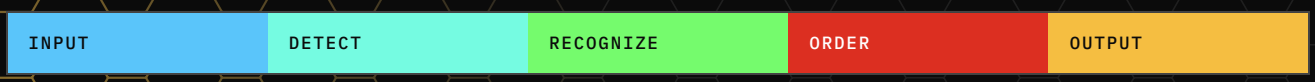
CPU and GPU · A worker per core · Many pages in flight · Every number measured

---

THE ENGINE GUIDE

How Carbon-OCR reads a page, and how to build and run it

The engine step by step: its inputs, orientation, detection, recognition, reading order and outputs; how it spreads its work over CPU cores, the backends and precisions it runs models in and the batched GPU pipeline; then building it on Windows, Linux and macOS, and using the command line, the HTTP server, the Python module and the OCRmyPDF plugin.



Karan Vora, Founder · Carbon Research

Contents

—	At a glance	3
I	**PART I · READING A PAGE**	4
01	A page's path	5
02	Inputs: images, PDFs and text layers	6
03	Orientation and deskew	7
04	Detection	8
05	Crops, recognition and CTC decoding	9
06	Splitting, reading order and tables	10
07	Outputs	11
II	**PART II · RUNNING FAST**	12
08	The CPU layout	13
09	Backends and the files they run	15
10	The GPU batched pipeline	17
11	Options	18
12	Measured so far	21
III	**PART III · BUILDING**	22
13	Requirements	23
14	The CPU build	23
15	The GPU build	24
16	Options	24
17	The Python module	25
18	Tests	26
IV	**PART IV · USING**	27
19	Reading pages	28
20	Serving pages over HTTP	29
21	Output formats	31
22	Timing and inspecting	31
23	Python	32
24	OCRmyPDF	32

At a glance

Pages in, text in reading order out, on every core or a GPU. Carbon-OCR turns a page right, levels it, finds its lines, reads every line in batches, and puts the page in reading order, with its blocks, labels and tables. A born-digital PDF page is read from its own text layer where that passes the checks.

INPUT	PDFs and page images · PDFium in helper processes, so a PDF that crashes it takes down a helper, not the caller · the text-layer checks, and hybrid pages
DETECT	orientation by quarter turns when sure · deskew up to 5 degrees · line and picture detection at 1024 × 800
RECOGNIZE	every line cropped to a 32-px band · CTC recognizers in batches · on the GPU, the CTC head fused with greedy decoding, so the logits never leave the chip
ORDER	lines split at gutters and between cells · the order model's reading order, blocks, labels and tables · the XY-cut where it is unsure
OUTPUT	text · markdown · HTML · JSON · hOCR · ALTO · PAGE XML · a searchable PDF

62 G

MAC/s for rec-latin on one CPU core: ONNX Runtime, FP32, AVX2, about 80% of the core's peak

98 ms

a Latin page on one CPU core, every model, the int8 files on the engine's own kernels

12.8 T

MAC/s for rec-latin in FP16 on a laptop RTX 4060: 256 lines in 5.1 ms on TensorRT 11.4

40 ms

one page alone on 6 cores, every output byte the same as on one

202

tests: 186 in CI on Windows, Linux and Arm, and 16 more on a GPU

STATUS, STATED PLAINLY

The engine is built and tested, on CPUs and NVIDIA GPUs. The first Fullerene set, fullerene-3M (Latin), is out on Hugging Face under the Carbon-OCR OpenRAIL-AMS license; the engine reads it as it ships. The files, metadata and precisions of a set are specified in the models document.

HOW THE NUMBERS ARE MEASURED

On a laptop (i7-13620H, RTX 4060 Laptop GPU), 27 to 30 October 2026, with random weights: speed does not depend on them. Every number is a measurement on that machine, never an estimate.



PART I

Reading a page

A page's path through the engine, from its pixels or its PDF to text in reading order.

01 A page's path

02 Inputs: images, PDFs and text layers

03 Orientation and deskew

04 Detection

05 Crops, recognition and CTC decoding

06 Splitting, reading order and tables

07 Outputs

01 A page's path

INPUT	PDFs and page images (PNG, JPEG, BMP, PNM) · PDFium in helper processes · the text-layer checks: born-digital pages that pass skip OCR, and only their pictures are read
DETECT	orient: 12 angle bins, quarter turns only when sure · deskew up to 5 degrees · det at 1024 × 800, maps at stride 2 · line boxes and pictures · tiny text detected again at twice the scale
RECOGNIZE	every line of the page cropped to 32 px · CTC recognizers, a frame per 4 px, batched; on the GPU, across the pages in flight, the CTC head fused with greedy decoding
ORDER	line and cell splitting · reading order and layout labels · tables' cells · the XY-cut where the model is unsure
OUTPUT	text · markdown · HTML · JSON · hOCR · ALTO · PAGE XML · searchable PDF · word boxes

On CPU, one single-threaded worker per core reads one page at a time. On GPU, many pages are in flight and lines are batched across pages.

STEP	WHAT HAPPENS	HEADER
input	an image decoded to gray, or a PDF page rendered in a helper process; a born-digital page whose text layer passes the checks is read from that layer instead	`image.h` , `pdf.h` , `textlayer.h`
scale	the page resampled once to the detector's scale (long side 1024 px), and a 512 × 512 thumbnail made from that copy	`image.h`
orient	`orient` reads the thumbnail; the page is turned by a quarter turn, and only when the model is sure	`engine.h`
deskew	a projection search levels what is left, up to 5°	`deskew.h`
detect	`det` reads the upright page; the line decoder turns its maps into line and picture boxes; a page of tiny text is detected again at twice the scale	`detect.h`
crop	each line cropped to 32 px high by the crop rule, the lines batched by width	`lines.h`
recognize	the recognizer gives class scores every 4 px; greedy CTC decoding gives the text, each character's position and probability	`lines.h`
split	lines split at wide gaps: column gutters, a table row's cells	`engine.h`
order	the order model's reading order, labeled blocks and table cells, or the XY-cut's order	`order.h`
output	text, JSON, hOCR, ALTO, PAGE XML, markdown, HTML or a searchable PDF	`output.h`

Two classes run these steps. `carbon::Engine` (`carbon/engine.h`) reads a list of pages on the CPU, in parallel, or a stream of them (`submit` and `next` , section 8). `carbon::Pipeline` (`carbon/pipeline.h`) reads a stream of pages with every model batched across the pages in flight, on the CPU or a GPU. Both use the same code for every

step, and on the CPU a page comes out of the pipeline exactly as it comes out of the engine. `carbon::Server` (`carbon/server.h`) puts either behind HTTP/1.1 (section 8). `carbon/carbon.h` includes the whole public API but the server's, which is a library of its own.

```
carbon-ocr ocr page.png scan.jpg --set models/fullerene-3M        # text, pages apart by \f
carbon-ocr ocr report.pdf --set models/fullerene-3M --format json  # boxes, words, blocks
carbon-ocr ocr scans/*.png --set models/fullerene-3M --device cuda --engine-cache .engines
```

```
import carbon_ocr

engine = carbon_ocr.Engine("models/fullerene-3M")   # a worker per physical core
page = engine.ocr([carbon_ocr.load_image("page.png")])[0]
print(page["text"])
```

Not in the engine yet. These parts of the design are still to come: the script classifier that routes each line to the recognizer of its script group (today a page is read by one recognizer, `EngineOptions::rec`), the second detection pass on a GPU (section 4), the language models that read unsure lines again, the path for vertical CJK columns, bidirectional reordering for right-to-left text, and TIFF input.

02 Inputs: images, PDFs and text layers

Images. `carbon::load_image` reads PNG, JPEG, BMP and PNM files and converts them to 8-bit gray, paper 255 and ink 0. It refuses an image more than 32,768 px on a side. Every later step works on gray pixels. A PNG goes through the engine's own decoder (`carbon/src/png.cpp`): libdeflate's inflate, the rows unfiltered with SIMD and turned gray as they come, giving stb_image's bytes about three times as fast; stb_image reads every other image, and any PNG the decoder would not read byte for byte as it does (interlaced files, for one).

PDFs. PDFium renders a PDF's pages and reads their text, and it runs only in helper processes. `carbon::PdfPool` starts `carbon-pdf` helpers, one per physical core by default, each with its own PDFium, and gives each one request at a time. A page's pixels come back through a shared memory region of the helper's (48 MiB, enough for a Letter or A4 page at 600 dpi); a larger page comes down the helper's pipe. PDFium is not thread-safe, so the helpers are how many pages render at once. A PDF that crashes PDFium takes down one helper, which is started again; the page it was on fails with `carbon::Error`, and the calling program carries on. A page is rendered as shown (its `/Rotate` applied, its annotations drawn, on white paper). `carbon-ocr ocr` renders PDF pages at `--dpi`, 200 by default.

Text layers. A born-digital page carries its own text. `carbon::read_text_layer` (`carbon/textlayer.h`) turns the characters PDFium reads into lines, and checks them before trusting them (`TextLayerRule`):

CHECK	THE LAYER IS NOT USED WHEN	DEFAULT
enough	the page draws fewer characters than `min_chars`: it is pictures or a scan	4
bad	more than `max_bad` of them have broken Unicode maps, or are U+FFFD, controls or private-use code points (legacy font encodings, symbol fonts)	2%
invisible	more than `max_invisible` are drawn neither filled nor stroked: an OCR layer over a scan, read again rather than trusted	0

CHECK	THE LAYER IS NOT USED WHEN	DEFAULT
no ink	more than `max_no_ink` have no pixel at gray 160 or darker under their box on the rendered page: white text, clipped text, glyphs that are not the codes they claim	5%

A page that passes skips detection and recognition. Its lines end at PDFium's line breaks and wherever the next character leaves the band of the one before. A line's box is the union of its characters' loose boxes (the font's whole height), its words' boxes are their characters', and its confidence is 1. Its lines are split at wide gaps and put in reading order like OCR's (section 6), and its `source` is `"text layer"`. A PDF page read by OCR says why its layer was not used (`text_layer` in the JSON).

Hybrid pages. A page read from its text layer can still hold text the layer does not: a chart's labels, a scanned table, a screenshot. Its image objects at least 32 px on both sides (`HybridRule::min_side`) are kept on an otherwise blank copy of the page, and that copy is read by OCR at the page's own scale. The lines found join the layer's, except those at least half covered by the layer's own lines (text drawn over a picture), and the page's `source` becomes `"hybrid"`. An image inside a form XObject, or a picture drawn as paths, is not seen.
`carbon-ocr ocr --text-layer off` reads every page by OCR, and `--hybrid off` leaves pictures unread.

03 Orientation and deskew

Each page is resampled once to the detector's scale, its long side 1024 px (`det_long_side`), with the crop rule's antialiased bilinear filter (section 5). Orientation reads a thumbnail of that copy, fitted into 512 × 512 (`orient_side`) at the top left, as an ink map (1 − gray / 255: ink 1, paper 0) with paper around it. The resampling runs on the CPU's vectors (AVX2 or NEON), the scalar code's bytes.

`orient` gives 13 class scores: 12 bins of 30° (bin k: the text turned 30k° clockwise) and one for vertical CJK columns. The engine turns a page only by a quarter turn, for the 90°, 180° and 270° bins, and only when that bin's softmax probability reaches the turn threshold. The threshold is `EngineOptions::turn_probability` when it is set, else the one stored in the set's orientation model (`carbon.orient.turn_probability`, see MODELS.md), else 0.95. Any other answer leaves the page as it is, since a wrong turn loses the whole page. `OcrPage::rotation` records the turn.

Deskew (`carbon/deskew.h`) levels what is left, without a model. On the page at the detector's scale, it finds the ink (Otsu's threshold), samples it to a fixed number of pixels and projects it onto the rows of the page turned by each candidate angle. The sum of the squared row counts is largest when the lines are level, because their ink falls in few rows and the gaps between them are empty. The search takes 0.5° steps within ±5° (`max_skew`), then 0.05° steps around the best. The page is levelled only when the angle is at least 0.2° (`min_skew`) and its score beats the level page's by 5% (`min_skew_gain` 1.05). `OcrPage::skew` records the angle. The turn runs in AVX2 or NEON doubles and the quarter turns by tile transposes, each the scalar code's bytes. With fast cores idle (a page alone), the search runs on the page as given beside orientation (a page orientation turns is searched again), and the levelling turns the page in row bands on the idle workers, each row the same bytes however the rows are cut.

A page's boxes are in the pixels of the page as turned upright and levelled. `carbon-ocr ocr --as-given` skips both steps, so the boxes are the given page's; the OCRmyPDF plugin uses it, because OCRmyPDF turns and levels pages itself.

04 Detection

`det` reads the upright page at the detector's scale as an ink map, padded with paper to a multiple of 32 (its largest stride), and gives four maps at stride 2:

MAP	VALUE
core	a logit: the line's core band, the middle 40% of its height, shortened at both ends by 0.15 of its height, so the bands of neighboring lines never touch
top	at a core pixel, the distance in input pixels from the pixel's center up to the line's top
bottom	the same, down to the line's bottom
picture	a logit: inside a picture

The detector's head on the CPU's vectors. The detector's last layers, its head (a 1 × 1 convolution and ReLU, a nearest 2× upsample, a depthwise 3 × 3 and ReLU, and a 1 × 1 convolution to the four maps), make two tensors at stride 2, 12.5 MB each for a Letter page in FP32, that only they read. When `carbon::Engine` opens the detector on the CPU (its FP32, FP16 or int8 file, on ONNX Runtime or its own kernels), it cuts the model before its head and computes the head itself (`cpu::det_head` , AVX2 or the scalar reference) from the stride-4 features, in row bands that stay in the cache, on the idle fast cores when there are some. Those two tensors are never written. The head's arithmetic is FP32 whatever the file's, and a row's maps are the same however the rows are cut.

The line decoder (`carbon::decode_det`) is C++, not a model. It thresholds the core and picture maps at a probability of 0.5, 8 pixels a compare on the CPU's vectors, and takes the 8-connected components of the core band (runs found a 32-pixel word at a time, joined by union-find). A line's top and bottom are the 5% and 95% quantiles of its band pixels' edges, so a slightly slanted line keeps both ends, and its ends lie 0.15 of its height beyond the band's. A band under 2 map pixels is noise, as is a picture under 16. A line's `det_score` is the mean probability over its pixels. The boxes are scaled back to the upright page's pixels, top to bottom, then left to right. `DetRule` holds all of these numbers.

On a GPU the threshold runs on the device (`cuda::threshold_det`) in three launches: a warp votes each 32-pixel word's bits, a scan gives every set pixel its place, and a scatter writes the set pixels' values. Only the bits and those values cross to the host, the same `DetMasks` that `carbon::threshold_det` makes on the CPU, and the host decodes them.

Detection on idle cores. When fast cores are idle, `carbon::Engine` detects a page on a shared multi-threaded ONNX Runtime session of `det.onnx` (2, 4 or 6 threads, the widest the idle fast cores fit) instead of the worker's own single-threaded one. `carbon/strips.h` offers a second way: the page's rows cut into strips that run side by side and are stitched back together. Each strip runs on its own rows plus 128 rows of the page above and below (the detector sees 127 px around a pixel), and every cut falls on a multiple of 32, so the stitched maps are the whole page's, byte for byte. The halos cost compute (8 strips of a 1024-row page run 2.5 times the rows), so strips pay only when many cores are idle. `carbon-ocr pages --det auto|one|wide|strips` times one against the other.

Tiny text: a second pass. At stride 2, a line under 9 px high at the detector's scale has a core band one or two map rows high, so the detector misses some such lines and places the rest coarsely. When the first pass's median line is under 9 px (`DetRule::second_pass_below`), the engine detects the upright page again at twice the scale (`second_pass_scale`), its long side at most 2048 px (`EngineOptions::second_pass_long_side`), where those

lines are about 18 px high. A page that large runs in strips of about a first pass's pixels, one after another (`carbon::run_in_strips`), so a worker's detector holds about a first pass's memory while the stitched maps stay the whole page's. On a page of ordinary print the second pass would only find the same lines again, and where the passes disagree it costs precision, so the median gates it.

The passes are merged line by line (`carbon::merge_passes`). A line of each pass whose boxes overlap by more than 0.5 IoU (`merge_iou`) is one line, kept from the pass whose scale suits it: the second's where its height is under 9 px at the first pass's scale, the first's otherwise. A line only one pass found is kept, one pass's miss being the other's find, except a first-pass box over text the second read as tiny lines (two or more of them mostly inside it, or one that fills most of it), whose text would otherwise be read twice. Pictures are merged the same way. `OcrPage::second_pass` records the pass and `ms.second_pass` its time; `carbon-ocr ocr --second-pass off` keeps one pass. The batched pipeline runs it on the CPU and on a GPU alike; on a GPU, whose first pass runs only at fixed sizes, its strips run on TensorRT at a profile of their own (section 10).

05 Crops, recognition and CTC decoding

The crop rule (`carbon::CropRule`, `carbon/lines.h`). A line box of height h becomes a crop 32 px high. The box gets h × 0.2 of padding above and below and h × 0.3 at each end, is scaled to 32 px, then is widened on both sides into the page to a multiple of 8, the recognizer's width step. The resampling is a triangle filter whose support widens with the downscale (an antialiased bilinear), in 22-bit fixed point: a horizontal pass into an 8-bit intermediate, then a vertical pass. Pixels outside the page read as paper. A crop of light text on a dark ground is flipped (255 − v), so a recognizer always reads dark ink on light paper: the test is whether more of its pixels are at least 48 levels lighter than its ground (the median of its border) than as much darker, counted on the CPU's vectors. Then the crop is padded to its batch's width with its own ground. The CUDA kernel (`cuda::crop_lines`, four launches for every line of every page) gives the same bytes as the CPU code, because the coefficients are computed in IEEE double precision without fused multiply-adds. A recognizer whose file carries its own crop rule (`carbon.crop.height`, `carbon.crop.pad_v`, `carbon.crop.pad_h`) sets those three values.

Width buckets. Lines are batched by width. A crop runs at the smallest bucket that holds it: 32, 48, 64, 88, 120, 160, 208, 272, 360, 472, 616, 808, 1056, 1376, 1792 or 2048 px, each at most 1.3 times the one before (`carbon::width_bucket`). A crop wider than 2048 px runs at its own width rounded up to 8. A run meets few shapes, and the padding stays small.

Recognition. The recognizer reads a batch of ink maps, `[N, 1, 32, W]`, and gives `[N, classes, W/4]`: a score for every class every 4 px of width, class 0 the CTC blank. Each line reads only the frames of its own crop width, not its bucket's.

CTC decoding (`carbon::ctc_greedy`). Greedy decoding takes the best class of each frame (ties go to the lower id), drops blanks and collapses repeats. Each character keeps the frame it starts at and its probability, the highest softmax probability over the frames of its run. On the CPU, `cpu::ctc_argmax` (AVX2 or NEON) reads a line's logits in their order, a class's frames after another's, keeping each frame's best so far in L1, then sums the softmax only for the frames where a character won, gathered 8 to a vector. Past a group's highest best class, each frame's sum is at least 1, so a class whose terms are all of logits more than 17 below their frames' bests, too small to change it, is passed over. The exponential is a polynomial in separate multiplies and adds, so every instruction set gives the same

bits. A line's text is its characters' strings from the vocabulary, and its confidence is their mean probability. A word is the characters between spaces; its box runs from the frame its first character starts on to the space after it, at the line's height.

The fused CTC head on the GPU. A recognizer's last layer, a 1×1 convolution to the classes, has its largest output: the logits of 256 lines of `rec-cjk` at 512 px are about 750 MB in FP16. On a GPU the engine runs the network up to that layer on TensorRT (`<rec>.features.fp16.onnx`). Then `cuda::ctc_head_argmax` computes the head on the tensor cores (FP16 in, FP32 accumulate) and reduces each frame to its best class and softmax probability as the logits are made, so they are never written. Each warp holds 16 frames of a line in registers while the weights stream through shared memory 32 classes at a time. Only ids and probabilities come back, `carbon::ctc_collapse` turns them into lines, and ties go to the lower id, as on the CPU. The kernel takes a head of 128, 192, 256 or 320 input channels. Without the trunk and head files, or at another precision (section 9), the GPU runs the whole recognizer and decodes its logits on the device (`cuda::ctc_greedy`).

06 Splitting, reading order and tables

Splitting (`carbon::split_lines` , `SplitRule`). A detector or a text layer can run one line across a column gutter or a table row's cells. After recognition, a line is split between two words where the gap is wide: at least 1.5 line heights (`always`), or at least 2.5 times the line's word spacing (`ratio`) and never under 0.6 heights (`min_gap`). The word spacing is the median of the line's gaps under 0.6 heights, or 0.25 heights where it has none. Each piece becomes a line of its own.

The order model. `order` reads 16 features for each line (its box, center, width, height and aspect against the page, and text cues such as a list marker or a sentence end) and gives pairwise scores for five relations and one of 13 block labels per line (MODELS.md). `carbon::decode_order` (`carbon/order.h`) turns them into an order: a greedy chain from the page's start over the successor probabilities, then up to 4 repair sweeps (`OrderRule::sweeps`) that move a line wherever the chain's log probability rises. Blocks are the runs of that order whose neighbors are in the same block (a pairwise probability of at least 0.5, `join`), each labeled by the mean of its lines' label probabilities. A table's lines join into cells, and cells into rows and columns, by the same rule, rows numbered top to bottom and columns left to right.

The XY-cut. When the chain's mean link probability is under 0.5 (`OrderRule::min_sure`), the engine takes the XY-cut's order instead, keeping the model's blocks over it. Without an order model (or with `use_order` false), the XY-cut alone orders the page (`carbon::reading_order`): the boxes are split again and again at the widest gap across them, a column gutter before the space between lines, then read top to bottom and left to right where no gap is left. `OcrPage::order` says which order a page has: `"model"` or `"xy-cut"` .

Tables from geometry. A page without the order model's blocks still gets its tables (`carbon::find_tables` , `TableRule`). Lines share a row where their heights overlap by half of the smaller's. A run of at least 2 rows, at most 2 line heights apart, each with 2 or more cells in columns of their own, is a table, unless its median cell holds more than 30 characters (columns of text, not a table). The page then gets blocks: each table a `Table` block with its cells, read row by row, and the other lines `Text` blocks, a new one where the gap between lines is wider than a line. These blocks have confidence 0, since no model made them.

07 Outputs

Each page comes out as a `carbon::OcrPage`: its size, the `rotation` and `skew` undone, its lines in reading order (box, text, confidence, `det_score`, words), its pictures, its blocks and their cells, its `source` and `order`, whether it took the second detection pass (`second_pass`), and its time per stage in ms (`orient`, which includes deskew, `detect`, `decode`, `second_pass`, `recognize`, `total`). `carbon/output.h` writes pages in the formats other tools read. Where a page has no blocks, it is written as one block of all its lines.

FORMAT	`--FORMAT`	WHAT
text	`text`	each page's lines in reading order, one line a row; pages apart by a form feed
JSON	`json`	`{"pages": [...]}`: per page its file and page number, size, rotation, skew, source, order, lines with words, pictures, blocks with cells, and timings
hOCR	`hocr`	hOCR 1.2 (XHTML), the format Tesseract writes and OCRmyPDF reads: `ocr_page`, then `ocr_carea` and `ocr_par` per block, `ocr_line` (`ocr_header` for a heading's line, `ocr_caption` for a caption's), `ocrx_word` with `x_wconf`
ALTO	`alto`	ALTO XML 4.4: `Page`, `PrintSpace`, `TextBlock` (its label a `LayoutTag`), `TextLine`, `String` (with `WC`) and `SP`
PAGE XML	`page`	PAGE XML (2019-07-15), one file a page in `--out-dir`: `TextRegion` (its type from the label), `TextLine`, `Word`, each with `Coords` and `TextEquiv`
markdown	`markdown`	headings, paragraphs, lists, tables as HTML from their cells, captions in italics; running headers and footers left out; pages apart by a rule
HTML	`html`	an HTML5 document, a `<section class="page">` per page: headings `<h2>`; paragraphs `<p>`, a block's lines joined and a word broken by a hyphen at a line's end joined again; lists `<ul>` and `<ol start>`; tables `<table>` from their cells; captions in `<figure><figcaption>`; running headers and footers `<header>` and `<footer>`, kept; footnotes, formulas and forms in elements of their own
searchable PDF	`pdf`	each page's image (8-bit gray) with its words over it as invisible text, each stretched to its box, in a font with no glyphs and a ToUnicode map; `--text-only` writes the text alone

A searchable PDF needs `--output FILE`. Its pages are a PDF's at `--dpi` and an image's at `--image-dpi` (300). `--text-out FILE` writes the plain text beside any format.

OCRmyPDF. `tools/ocrmypdf/carbon_ocrmypdf.py` makes Carbon-OCR the OCR engine of OCRmyPDF 17. For each page OCRmyPDF rasterizes, it runs `carbon-ocr ocr --as-given` and takes hOCR, or, with `--pdf-renderer sandwich`, a text-only PDF to graft onto the page. Its options are `--carbon-set DIR` (else `$CARBON_OCR_SET`), `--carbon-ocr PATH` (else `$CARBON_OCR`, else `PATH`) and `--carbon-workers N` (1 by default, since OCRmyPDF runs pages in parallel itself). It reads the Latin-script languages `rec-latin` covers, named by Tesseract's codes. OCRmyPDF's built-in Tesseract plugin still checks that `tesseract` is installed.

```
ocrmypdf --plugin tools/ocrmypdf/carbon_ocrmypdf.py \
    --carbon-set models/fullerene-3M in.pdf out.pdf
```



PART II

Running fast

The cores, the backends and the GPU: how the engine spreads its work, and what it measured.

08 The CPU layout

09 Backends and the files they run

10 The GPU batched pipeline

11 Options

12 Measured so far

08 The CPU layout

A pinned worker per physical core (`carbon::WorkerPool`, `carbon/parallel.h`). The pool has one thread per physical core, not per hardware thread (a core's two threads share its FMA units), each pinned to its core. On Windows the workers take a class of cores in the order of the cores' scheduling class, the favored (highest-clocked) cores first, and the core that takes the most interrupts last. Each worker opens its own single-threaded session of every model on its own thread, so its memory sits in its core's NUMA node and running a model never takes a lock. The sessions run with one thread each, so the pool is the only parallelism and the OS never juggles more threads than there are cores; the one exception is detection on idle fast cores (section 4). Idle workers sleep rather than spin.

`carbon::cpu_topology()` reads the machine once (`GetLogicalProcessorInformationEx` on Windows, `/sys` and the cgroup on Linux). It finds the physical cores; Windows processor groups, so a machine with more than 64 logical processors is used whole; the NUMA nodes, which workers alternate between; P-cores before E-cores on hybrid CPUs; and the process's affinity, a container's cpuset and its CPU quota. `carbon-ocr cpus` prints what it found.

Tasks smaller than a page. A page starts as one task: orientation, deskew, detection and the line decoder. Its lines then become tasks of their own: one per idle fast core, of about equal cost (a line costs its width bucket's width, so the tasks finish together), or 8 lines a task (`line_batch`) when every core is busy. The last line task to finish splits the page's lines and orders them. A worker runs the tasks it made newest first; with none left, it takes the oldest task of another worker before it starts a new page. So a large job runs about a page per worker, a few pages spread over every core, and the pages in flight finish first. Results come back in the order the pages were given.

Fast cores first. On a hybrid CPU, a worker on an E-core takes a task only while every P-core worker is running one, and only the tasks of pages already started (their lines), never a new page: a page's detection on an E-core runs 1.5 times as long and holds the page up. A lone page runs on the P-cores, and a full load on every core.

A stream of pages. `Engine::submit` gives the engine a page and returns its number, from 0 in the order given, and `Engine::next` returns each page's result in that order as soon as it is done, waiting for it (and rethrowing the error that page met; the stream goes on). Each page starts as soon as a worker is free, so a caller that streams pages, a server say, keeps every core busy with no wait for a batch to end. At most `in_flight` pages are being read at once (0: two a worker), past which `submit` waits; one thread may submit while another takes. `submit(file)` leaves the file's read and decode to the worker that takes the page (`ms.load`). The pages come out with the same bytes as from `ocr()`.

Pages as they finish. `Engine::submit_encoded(bytes, keep)` takes a page's encoded image (PNG, JPEG, BMP or PNM) and leaves its decode to the worker that takes the page, as `submit(file)` does; `keep` holds the bytes until then. `Engine::next_finished` gives the pages in the order they finish instead of the order given, each with its number and its result or the error it met (`unreadable` where its bytes or file did not decode), so a slow page holds back no other. A stream takes its results by `next` or by `next_finished`, not both.

The server (`carbon::Server`, `carbon/server.h`; `carbon-ocr serve`). It keeps a set's models loaded and reads pages sent over HTTP/1.1: `POST /ocr` with a page's image as the body, answered in any format of section 7 by the same writers as `carbon-ocr ocr`, and `GET /health` and `GET /info` ("Serving pages over HTTP" in USAGE.md). Three threads do the server's own work: one polls the listening socket and every connection (`WSAPoll` on Windows, `poll` elsewhere), one gives the pages to the engine's stream (`submit_encoded`), and one takes them as they finish (`next_finished`) and writes their answers. The engine's workers do the rest, so every client's pages share them as

one stream's would. With `cuda` set, a few threads of their own (`decoders`) decode the pages for the batched GPU pipeline instead (section 10), and every client's pages share its batches. Requests past the pages in flight wait in a queue of `queue` (64); past it, the server answers 503 with `Retry-After: 1` and goes on. A page that fails is answered with its error (422 where its image does not decode, 500 otherwise), and the pages after it are read as ever, on either device. The server is a library of its own (`carbon_serve` , `carbon::serve`), so the engine library and the Python module carry no sockets, and `carbon-ocr ocr` reads as before.

Memory kept. A tensor of 256 KB or more takes its memory from a cache kept by each thread (up to 48 MB), where the blocks of the tensors it made before wait, instead of a fresh block from the system: on Windows a fresh block is first touched at about half a millisecond a megabyte, and a page makes tens of megabytes of them. The recognizer's logits and the detector's features are read where ONNX Runtime wrote them, not copied.

The engine's own kernels (`carbon/cpu.h`). Each kernel has a scalar reference and a version per instruction set, chosen at run time from what the CPU reports (`cpuid` and `xgetbv` on x86-64, the OS on Arm), so one binary runs at its best on Intel, AMD and Arm:

NAME	INSTRUCTIONS	CPUS
`avx2`	AVX2 and FMA: FP32, int8 through exact 16-bit products (`vpmaddwd`), FP16 widened by F16C (`vcvtph2ps`)	AMD Zen 2 and 3, Intel before Ice Lake
`avx-vnni`	AVX-VNNI: `vpdpbusd` on 256 bits	Intel Alder Lake and later clients, AMD Zen 5
`avx512`	AVX-512 F, BW, VL, DQ and VNNI: `vpdpbusd` on 512 bits	Intel Ice Lake and later servers, AMD Zen 4 and 5
`avx512-fp16`	AVX-512 FP16: FP16 arithmetic (`vfmadd231ph` , 32 halves a vector)	Intel Sapphire Rapids and later servers
`neon`	Armv8-A Advanced SIMD: FP32, FP16 widened (`fcvtl`); int8 by the scalar reference	every Armv8-A
`neon-dot`	the dot product extension (`sdot`)	Armv8.2-A and later: Graviton 2 to 4, Apple M, Neoverse, Cortex-A76 on
`neon-fp16`	dotprod and FP16 arithmetic (`fmla` on 8 halves)	the same CPUs, but a few

The kernels are an int8 GEMM (uint8 activations with a zero point, int8 weights packed once per instruction set), an FP32 GEMM, an FP16 GEMM and depthwise convolution, a depthwise convolution and CTC's argmax; and the page's own work: the crop rule's resampling (each output's taps as exact 32-bit integer products, the scalar code's bytes), the deskew's turn (bilinear in doubles, each operation rounded as the scalar code's), the quarter turns (tile transposes in registers) and PNG's row filters. A convolution reads its input in place, each row of A a list of pieces, one per tap of the kernel, so nothing is copied into columns (no im2col). The output stage runs inside the kernel while the sums are still in L1: scale, bias, a float or uint8 residual, ReLU, then out as float or requantized uint8 (rounded half to even). Every version gives the scalar reference's bits. Integer sums are exact, a float sum is one fused multiply-add after another in the order of k, and the output stage is the same float operations in the same order everywhere, so a page reads the same on Intel, AMD and Arm. Each instruction set's code is compiled with its own flags in its own file, and the rest of the library stays baseline x86-64 or Armv8-A. `CARBON_ISA=<name>` (or `SessionOptions::isa`) picks another set for tests and timing, and `carbon-ocr cpus --gemm` times the GEMM with each one.

09 Backends and the files they run

Every model runs through `carbon::Session` (`carbon/session.h`), so a step never knows which backend runs its model. `carbon-ocr version` lists the backends a build has.

BACKEND	NAME	RUNS	WHERE
native	`native`	the convolutional networks from their ONNX files, int8 (Q/DQ), FP16 or FP32; `order` from `order.native.bin`	every build, on the CPU
ONNX Runtime	`ort`	any ONNX file on the CPU; the engine gives it the FP32 files	builds with ONNX Runtime (1.31.0, pinned)
TensorRT	`trt`	an ONNX file in its own precision: TensorRT 11 builds strongly typed networks, so an FP16 file runs in FP16 and a Q/DQ file in INT8	builds with CUDA 12.9 or 13.x and TensorRT 11, on an NVIDIA GPU

The native backend. It reads the ONNX file itself and compiles the nodes into fused layers: a convolution takes the ReLU after it and a residual Add, so its output stage does the bias, the residual and the ReLU while the sums are in L1. Its layers are Conv (dense or depthwise), Resize (nearest, whole scales), AveragePool, ReduceMean over the image, Gemm and Add, and it refuses any other op when the file is opened. Every tensor is NHWC with a border of zeros as wide as its readers' padding, so an output row of a convolution is one GEMM over a grid of its input. A Q/DQ file runs in int8: a first pass reads the QuantizeLinear and DequantizeLinear nodes away into int8 weights with per-channel scales and a uint8 scale and zero point for each tensor kept in uint8. Convolutions on uint8 inputs then run on the int8 kernels, and a residual Add joins its convolution in float, quantized once. `order`, a transformer, runs natively from `order.native.bin`: its weights, and its forward pass written out in C++ on the FP32 GEMM. `CARBON_NATIVE_PROFILE=1` prints each layer's time, and `carbon-ocr compare MODEL` runs a file on the native backend and on ONNX Runtime and prints how far apart their outputs are.

Which file, on which backend (`carbon::open_set_model`, `carbon::choose_set_model`). With no backend given, the engine picks per model:

MODEL	PICKED	BACKEND = NATIVE	BACKEND = ORT
`orient`, `det`, the recognizer	native on `<m>.int8.onnx` where the set has it; else ONNX Runtime on `<m>.onnx` where it is built in; else native on `<m>.onnx`	native on `<m>.int8.onnx`, else `<m>.onnx`	ONNX Runtime on `<m>.onnx`
`order`	native on `order.native.bin`; else ONNX Runtime on `order.onnx`	the same	ONNX Runtime on `order.onnx`
`det` on idle cores	ONNX Runtime on `det.onnx`, 2, 4 or 6 threads	not used	the same

Precision (`EngineOptions::precision`, `--precision`, `precision=` in Python): `auto` is the table above; `fp32` runs `<m>.onnx`, `fp16` `<m>.fp16.onnx` on the native backend and `int8` `<m>.int8.onnx`, each an error where the set lacks the file. An FP16 file's tensors are float32 at the session: where the CPU has no FP16 arithmetic (AVX2 with F16C, plain NEON) a layer's weights are widened to FP32 once a run and summed as FP32 (the FP32 kernels' bits), so FP16 runs at native FP32's speed with half the weights' memory; with FP16 arithmetic (AVX-512 FP16, Armv8.2 FP16) products are summed in FP16 at most 64 at a time, then in FP32, and the activations stay FP16 between layers. `order` stays FP32.

On the GPU (`carbon::gpu_set_model`), TensorRT's networks are strongly typed, so the precision picks the file it builds: `fp16` `<m>.fp16.onnx` , `fp32` `<m>.onnx` and `int8` `<m>.int8-sym.onnx` (symmetric Q/DQ), each an error where the set lacks the file; `auto` the FP16 file where the set has it, else the FP32 one. The recognizer at `auto` and `fp16` runs its trunk and the fused CTC head (section 5) from `<rec>.features.fp16.onnx` and `<rec>.head.bin` (at `fp16` , an error where the set lacks them). At `int8` it runs `<rec>.int8-sym.onnx` whole, and otherwise (`fp32` , `auto` without those files, or `fused_head` off) `<rec>.onnx` whole, its logits decoded on the device. The stages' buffers and kernels take each model's inputs and outputs in FP16 or FP32, as its file has them. `fullerene-3M` ships its FP32 and FP16 files, so on a GPU it runs at `auto` , `fp16` and `fp32` .

`EngineOptions::order_model` names another order model: a `.bin` file runs natively, an ONNX file on ONNX Runtime. On one core the native int8 recognizer runs 2.5 times as fast as ONNX Runtime in FP32, and a native FP32 run is slower than ONNX Runtime's, so the int8 files are the CPU's path. ONNX Runtime sessions keep their memory arena on (`SessionOptions::cpu_arena`): a session reuses its runs' activation memory, which grows to the largest batch it has run, a size the width buckets bound.

TensorRT. A model with free dimensions needs optimization profiles, each a range for every free input. Each profile gets an execution context of its own, so a run never switches profiles, and a run takes the first profile exact for its shapes, else the first that holds them. A profile whose ranges are all exact is a bucket: TensorRT tunes for its one shape, and its runs are captured as CUDA graphs on the first run and replayed after (a graph per set of buffer addresses, up to 8). `Session::enqueue` runs on device buffers, queued on the caller's stream, and returns without waiting.

Engine caches. Building a TensorRT engine takes minutes. With `SessionOptions::engine_cache` (`--engine-cache DIR`), built engines are kept on disk, keyed by the model's bytes, the profiles, the build settings (workspace and optimization level), the GPU (its name and compute capability) and TensorRT's version, beside a timing cache that makes rebuilds quicker. A key that changes builds a new engine; without a cache directory, every open builds. `optimization_level` (0 to 5, default 3) trades build time for kernel search, and `cuda_graphs` turns the graph replay off.

10 The GPU batched pipeline

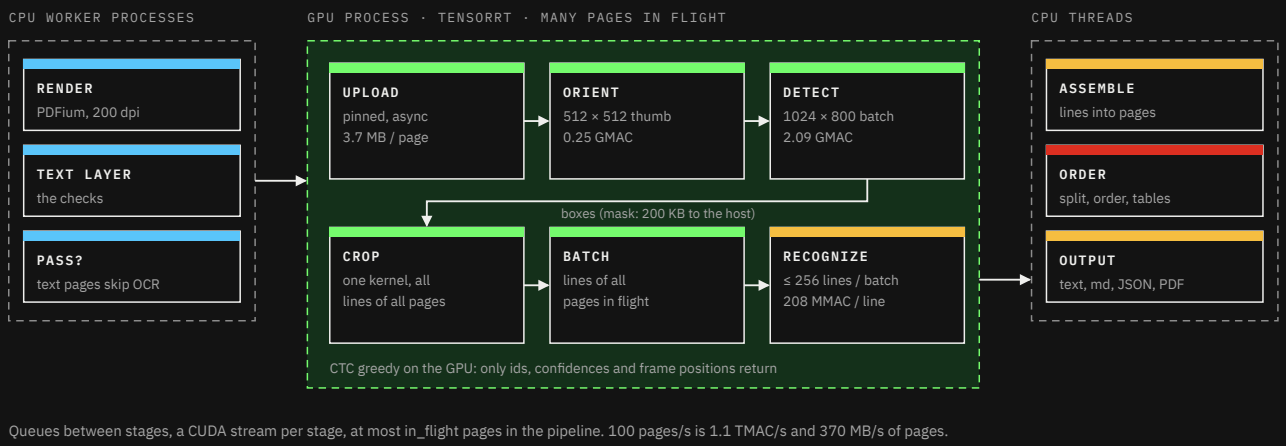


Figure 1. The GPU data flow: CPU worker processes render and check pages; the GPU process runs orientation, detection, cropping, routing and recognition for every page in flight; CPU threads assemble the output. Only ids, confidences and frame positions come back from the device.

`carbon::Pipeline` (`carbon/pipeline.h`) takes pages with `submit` and gives their results with `next`, in the order given, while its stages work on the pages in flight. At most `in_flight` pages (32) are in it at once, and `submit` waits for room. Each model stage is a thread of its own, on a GPU with a CUDA stream of its own, joined to the others by queues, so orientation, detection and recognition of different pages run at the same time. `ocr(pages)` does it all for a list. The figure shows the whole design; script routing and the language model's beam are not in the engine yet.

STAGE	RUNS ON	WHAT
prepare	CPU threads	each page at the detector's scale, and its thumbnail
orient	a thread and a stream	up to `page_batch` thumbnails in one run, made the file's FP16 or FP32 on the device; the turn each page asks for
upright	CPU threads	the quarter turn and deskew; the detector's padded size
detect	a thread and a stream	up to `page_batch` pages of one padded size in one run, uploaded as uint8 and made an ink map on the device, in the file's FP16 or FP32 (`cuda::ink_map`); the maps thresholded on the device and only the masks decoded on the host; a page of tiny text's second pass, its strips at a profile of their own
recognize	a thread and a stream	every line of the pages waiting, from any of them, by width bucket, up to `line_batch` a run: the pages uploaded once for the group, every line cropped in one set of launches, the trunk and the fused CTC head (or the whole recognizer and `cuda::ctc_greedy`); only ids and probabilities come back; a line wider than the widest bucket read by the CPU's recognizer
assemble	CPU threads	lines split, the order model (on the CPU) or the XY-cut, blocks and tables

Exact shapes. On a GPU every model runs on TensorRT from the set's files at the engine's precision (section 9), the FP16 files by default. Detection pads each page to one of three sizes at a long side of 1024: 1024 high by 800 wide (portrait), 800 high by 1024 wide (landscape), or 1024 square. Pages batch only with pages of the same size, in powers of two up to `page_batch` (1, 2, 4 and 8 by default). Each size and batch is an exact profile, and so a CUDA graph. Orientation has an exact profile per batch size too. The recognizer has a profile per width bucket, 32 to 2048 px, for batches of 1 to `line_batch` (256). Every buffer a model reads or writes is allocated at open at its largest, so its address stays fixed and the graphs replay. A page goes to the device as its thumbnail, its copy at the detector's scale and its upright self, each once and the last two as uint8 pixels; only detection's masks and recognition's ids and probabilities come back.

Past the fixed shapes. A page of tiny text takes the second detection pass as the CPU engine does (section 4): the same trigger, resample, strips and merge, the strips running on a session of `det` 's file of their own at one profile that holds every size they come in (to `second_pass_long_side` square), so the first pass's exact profiles and their cached engines stay as they are. A line whose crop is wider than the widest bucket (2048 px) has no profile; it goes to the CPU device's recognizer, opened at the first one, so the GPU reads every line the CPU reads.

Errors. A page that meets an error, in any step of its own or in a model run that held it, comes out with that error: `next` throws it and the stream goes on, as `Engine::next` does, on either device. Every page of a failed run fails with it, and `ocr(pages)` takes every page through before it throws the first page's error, so the pipeline takes the next call. `Pipeline(set, options, models)` takes models of the caller's own (`PipelineModels`): a device of its own, or a wrapper of `cpu_pipeline_models` or `cuda_pipeline_models` , as the tests' wrapper that fails each kind of run.

On the CPU (`Device::kCpu`), the same stages run the set's files as the CPU engine opens them, each model run on `model_threads` threads, and every page's result is the CPU engine's. With no `device` given, the pipeline runs on a GPU where the build has CUDA and TensorRT and a GPU is present (`carbon::cuda_pipeline_available()`), else on the CPU. `carbon-ocr ocr --device cuda` reads its pages through the pipeline on the GPU (`--precision` picks the files, `--batch` sets `page_batch` , `--in-flight` and `--engine-cache` the others), with PDF pages rendered by the helpers as before, and `carbon-ocr serve --device cuda` puts every client's pages in its batches (section 8). `stats()` counts the runs and what they held. The Python module's `Pipeline` runs on the CPU, because the module is built without TensorRT for now.

11 Options

`carbon::EngineOptions` (`carbon/engine.h`):

FIELD	DEFAULT	WHAT
`pool`	one worker per physical core, pinned	`PoolOptions` : `workers` (0: one per physical core, at most a CPU quota), `pin` , `fast` (the workers that count as fast; -1: those on the fastest class of core)
`backend`	none: per model, as in section 9	one backend for every model: `kNative` or `kOnnxRuntime`
`precision`	`kAuto`	the files the models run, on the CPU and the GPU: `kAuto` , `kFp32` , `kFp16` or `kInt8` (section 9)

FIELD	DEFAULT	WHAT
`rec`	`"rec-latin"`	the recognizer: `<set>/<rec>.onnx` and `<rec>.vocab.json`
`det_long_side`	1024	the page's long side at detection, padded to a multiple of 32
`second_pass_long_side`	2048	the page's long side at most, in the second detection pass
`orient_side`	512	the side of the thumbnail orientation reads
`turn_probability`	none: the set's own, else 0.95	how sure orientation must be to turn a page
`deskew`, `max_skew`, `min_skew`, `min_skew_gain`	true, 5.0°, 0.2°, 1.05	deskew, and when it levels a page
`line_batch`	8	lines a recognition task holds when every core is busy
`in_flight`	0: two a worker	pages the stream (`submit`) reads at once, past which `submit` waits
`use_order`, `order_model`	true, empty	the order model, or another file in its place; without one, the XY-cut
`order`	`OrderRule`	`min_sure` 0.5, `sweeps` 4, `join` 0.5
`split`	`SplitRule`	`min_gap` 0.6, `ratio` 2.5, `space` 0.25, `always` 1.5 (line heights); none: no splitting
`tables`	`TableRule`	`min_rows` 2, `min_cols` 2, `row_overlap` 0.5, `max_row_gap` 2.0, `max_cell_chars` 30; none: no tables from geometry
`det`	`DetRule`	`core_threshold` and `picture_threshold` 0.5, `min_core_pixels` 2, `min_picture_pixels` 16, `edge_quantile` 0.05; the second pass: `second_pass_below` 9 px (0: never), `second_pass_scale` 2, `merge_iou` 0.5
`crop`	`CropRule`	`height` 32, `width_step` 8, `frame_px` 4, `pad_v` 0.2, `pad_h` 0.3, `flip_margin` 48; a recognizer's own `carbon.crop.*` values set `height`, `pad_v` and `pad_h`

`carbon::PipelineOptions` (`carbon/pipeline.h`) adds:

FIELD	DEFAULT	WHAT
`engine`	`EngineOptions{}`	the recognizer, crop rule, orientation, deskew, order, splitting and tables; `engine.backend` for the CPU device
`device`	none: the GPU where available, else the CPU	`Device::kCpu` or `Device::kCuda`
`page_batch`	8	pages an orientation or detection run holds at most
`line_batch`	256	lines a recognition run holds at most
`in_flight`	32	pages in the pipeline at once
`threads`, `model_threads`	0, 0	CPU threads preparing and assembling pages, and a CPU model run's threads; 0: the physical cores
`gpu`	0	the CUDA device

FIELD	DEFAULT	WHAT
`engine_cache`	empty	where TensorRT's built engines are kept; empty: built at every open
`fused_head`	true	at `kAuto` and `kFp16`, the recognizer's trunk and the fused CTC head where the set has `<rec>.features.fp16.onnx` and `<rec>.head.bin`; off, the whole recognizer in FP32

`carbon::ServerOptions` (`carbon/server.h`) sets the server:

FIELD	DEFAULT	WHAT
`host`, `port`	`"127.0.0.1"`, 8080	where it listens: `0.0.0.0` for every interface; port 0 a free one (`Server::port()`)
`max_body`	64 MB	a request's body, past which 413
`queue`	64	requests waiting past the pages in flight, past which 503
`engine`	`EngineOptions{}`	the engine's options, as above
`cuda`, `pipeline`	false, `PipelineOptions{}`	the batched pipeline on the GPU, with `pipeline`'s options (its engine options are `engine`)
`decoders`	0: 4, at most the cores	on a GPU, the threads that decode the pages
`image_dpi`	300	`format=pdf`: the page's image at this dpi
`log`	none	a line a request, called on several threads

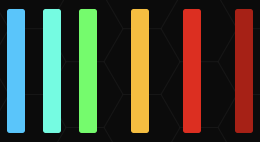
`Server::run()` serves until `Server::stop()`, which any thread or a signal handler may call: then no more requests are read, the pages being read are finished, and every connection is closed.

The text layer's checks are `TextLayerRule` (`min_chars` 4, `max_bad` 0.02, `max_invisible` 0, `max_no_ink` 0.05, `ink` 160) and hybrid pages' are `HybridRule` (`min_side` 32, `max_covered` 0.5), both in `carbon/textlayer.h`. `SessionOptions` (`carbon/session.h`) sets a single model's `threads`, the CPUs those threads keep to (`thread_cpus`), `cpu_arena`, `isa`, the model's bytes to run in place of its file's (`model_bytes`, as the detector cut before its head), and for TensorRT its `device`, `profiles`, `engine_cache`, `workspace_bytes` (1 GiB), `optimization_level` and `cuda_graphs`. Two environment variables matter: `CARBON_ISA` picks the kernels' instruction set, and `CARBON_NATIVE_PROFILE=1` times the native backend's layers. A `carbon::Engine`'s `ocr` is called from one thread, and its stream from one thread that submits and one that takes; a `carbon::Session` is not safe to run from two threads at once, so each worker has its own.

12 Measured so far

On a laptop with an Intel Core i7-13620H (6 P-cores, 4 E-cores) and an RTX 4060 Laptop GPU, 27 to 30 October 2026, with random weights: speed does not depend on them. The page timings are `carbon-ocr pages`, which runs a set's files the way the CPU engine runs a page.

WHAT	MEASURED
the trained `fullerene-3M`, end to end	OmniParseBench, all 2,937 pages: a headline of 43.5 against Tesseract 5.5's 5.9; 4.06 pages a second on one core against 0.76, 13.6 in six processes against 3.2 (3 November; FP32 on ONNX Runtime, the engine's HTML and layout blocks graded)
the trained `fullerene-3M`, in one process	10 workers with 20 pages in flight (`Engine::submit` and `next`): 15.0 pages a second over all 2,937 pages, 15.4 on 300 of them; one page at a time on 6 workers, 10.4 pages a second, the median page 79 ms; every page the same bytes as in six processes
`fullerene-3M` served over HTTP	`carbon-ocr serve` on the CPU, 48 OmniParseBench pages sent by 16 to 32 clients at once and answered as JSON: 18.5 pages a second, the images' decoding included; `carbon-ocr ocr` on the same pages 19.0, its count without the decoding (4 November)
`fullerene-3M` on the GPU	all 2,937 OmniParseBench pages through `carbon-ocr serve --device cuda` on the laptop's RTX 4060 (its FP16 files on TensorRT, pages of tiny text detected again, 32 pages in flight, detection batched 2 pages at most): 103.7 pages a second, every page read, a headline of 43.4 against the CPU's 43.5 (4 November)
`fullerene-3M` on olmOCR-Bench and OmniDocBench	each by its own grader against Tesseract 5.5, the pages read as images: olmOCR-Bench's 1,403 PDFs 41.4 ± 1.0 to 34.2 (tables 40.2 to 0.5, running headers and footers left out 72.2 to 40.1; Tesseract ahead on long tiny text, 60.6 to 54.5, multi-column pages, 52.3 to 48.6, and old scans, 20.7 to 15.2; math 0 for both); OmniDocBench v1.6's 755 English pages, text edit distance 0.231 to Tesseract's 0.129, reading order 0.355 to 0.253, tables 28.0 TEDS to none; six processes on the CPU, 12.7 pages a second to Tesseract's 3.0 (4 November)
`rec-latin` on one CPU core	8 lines of 512 px in 11.4 ms in int8 on the engine's own kernels; 16 lines in 65 ms on ONNX Runtime 1.31 in FP32 (AVX2), 62 GMAC/s, about 80% of the core's peak
`fullerene-3M` on one CPU core	a Latin page in 98 ms, every model included, the set's int8 files on the engine's own kernels: 10.2 pages a second per core
`fullerene-10M` on one CPU core	a CJK page in 271 ms
across CPU cores	no loss per clock cycle up to 4 workers; 47.4 pages a second on 10 workers, where the laptop's power limit lowers its clock as cores load
one page alone	40 ms on the 6 P-cores, 129 ms on one
`rec-latin` on the GPU	256 lines of 512 px on device buffers in 5.1 ms in FP16 (12.8 TMAC/s), 13.6 ms in FP32 and 2.98 ms in INT8 (TensorRT 11.4, CUDA 13.4.1)
the fused CTC head	1.6 to 2.1 times as fast as the cuBLAS GEMM and argmax it replaces, and it never writes the logits
CUDA graphs	one 224-px line in 0.164 ms instead of 0.242 ms



PART III

Building

Windows, Linux and macOS; the CPU and the GPU; the options; the Python module; the tests.

13 Requirements

14 The CPU build

15 The GPU build

16 Options

17 The Python module

18 Tests

13 Requirements

WHAT	VERSION
CMake	3.25 or newer (the presets need it)
a C++20 compiler	MSVC 2022, GCC 11+ or Clang 14+
Ninja	for the `ninja-*` presets
CUDA	12.9 or 13.x, only for the GPU build
TensorRT	11, matching the CUDA, only for the GPU build
Python	3.12, only for the Python module

CMake downloads these and checks each one's SHA-256:

DEPENDENCY	PINNED	USED FOR
ONNX Runtime	1.31.0	the CPU backend that runs any ONNX model
PDFium	chromium/8076	reading PDFs, in the `carbon-pdf` helper processes
libdeflate	1.24	the searchable PDF's images
stb_image	2.30	reading PNG, JPEG, BMP and PNM pages
GoogleTest	1.18.0	the C++ tests

ONNX Runtime downloads exist for Windows x64, Linux x64 and macOS arm64. On any other platform, set `ONNXRUNTIME_ROOT` to an unpacked release, or build with `-DCARBON_WITH_ONNXRUNTIME=OFF`; the engine's own int8 backend still runs. PDFium downloads exist for Windows x64, Linux x64 and arm64, and macOS arm64. Elsewhere, set `PDFIUM_ROOT` to an unpacked build, or turn PDFs off with `-DCARBON_WITH_PDFIUM=OFF`.

14 The CPU build

```
# Linux, macOS, or a Visual Studio developer shell
cmake --preset ninja-release
cmake --build --preset ninja-release
ctest --preset ninja-release

# Windows, Visual Studio 2022
cmake --preset vs2022
cmake --build --preset vs2022
ctest --preset vs2022
```

The build goes to `build/<preset>/`, and the programs are in `build/<preset>/cli/`:

- `carbon-ocr`, the command line.

- `carbon-pdf`, the helper that PDFs are rendered in. It must stay beside `carbon-ocr`, because the engine looks for it there.

On Windows, the DLLs both programs need are copied next to them.

The CPU kernels are compiled once for each instruction set the compiler supports, and the engine picks one at run time: AVX2, AVX-VNNI and AVX-512 on x86-64; NEON and the dot product on Arm. The rest of the library stays at the baseline (x86-64 or Armv8-A), so one binary runs on any CPU of its architecture. To see the kernels a machine runs, use `carbon-ocr cpus`.

15 The GPU build

1. Install or unpack CUDA 12.9 or 13.x.
2. Unpack a TensorRT 11 release that matches the CUDA, and set `TensorRT_ROOT` to that folder, either as an environment variable or as a CMake cache variable.
3. Put TensorRT's and CUDA's library folders on `PATH` (Windows) or `LD_LIBRARY_PATH` (Linux).
4. Configure with the `ninja-trt` preset (on Windows, in a Visual Studio developer shell) or the `vs2022-trt` preset:

```
cmake --preset ninja-trt -DCARBON_WITH_CUDA=ON -DCMAKE_CUDA_ARCHITECTURES=89
cmake --build --preset ninja-trt
ctest --preset ninja-trt
```

`CMAKE_CUDA_ARCHITECTURES` defaults to `native`, which builds for the GPU in the machine you build on. Set it to build for others: 89 for Ada (an L4, an RTX 40-series card), 86 or 80 for Ampere. With a CUDA that is unpacked rather than installed, also pass `-DCMAKE_CUDA_COMPILER=<cuda>/bin/nvcc` and `-DCUDAToolkit_ROOT=<cuda>`. The `ninja-cuda` preset builds the CUDA kernels without TensorRT.

TensorRT 11 builds strongly typed networks, so the ONNX file is the precision: a set ships `<model>.fp16.onnx`, and can ship `<model>.int8-sym.onnx`, beside each FP32 file, and `--precision` picks among them (ENGINE.md, section 9). Building a TensorRT engine takes time; `--engine-cache DIR` keeps the built engines, so later runs reuse them. A cached engine belongs to its GPU and its TensorRT version.

16 Options

OPTION	DEFAULT	WHAT
`CARBON_WITH_ONNXRUNTIME`	ON	the ONNX Runtime backend (CPU)
`CARBON_WITH_TENSORRT`	OFF	the TensorRT backend (GPU); needs `TensorRT_ROOT`
`CARBON_WITH_CUDA`	OFF	the CUDA kernels: the crop rule, CTC decoding, the fused CTC head, the detector's threshold; with TensorRT, the batched GPU pipeline

OPTION	DEFAULT	WHAT
`CARBON_WITH_PDFIUM`	ON	PDF input, through PDFium in helper processes
`CARBON_BUILD_CLI`	ON	`carbon-ocr` and `carbon-pdf`
`CARBON_BUILD_TESTS`	ON when this is the top-level project	the C++ tests
`CARBON_BUILD_PYTHON`	OFF	the Python module (`pip install ./python` turns it on)
`ONNXRUNTIME_ROOT`	empty: download	an unpacked ONNX Runtime release
`PDFIUM_ROOT`	empty: download	an unpacked PDFium build
`TensorRT_ROOT`	the environment variable	an unpacked TensorRT release

PRESET	WHAT
`ninja-release` , `ninja-debug`	Ninja, CPU
`vs2022`	Windows, Visual Studio 2022, CPU
`ninja-trt` , `vs2022-trt`	CPU and GPU (TensorRT)
`ninja-cuda`	CPU and the CUDA kernels, without TensorRT

To use the library from another CMake project, add this repository with `add_subdirectory` (or FetchContent) and link `carbon::carbon` . Its public headers are in `carbon/include/carbon/` , and `carbon/carbon.h` includes them all but the server's: for `carbon::Server` (`carbon/server.h`), link `carbon::serve` too.

17 The Python module

```
pip install ./python                                    # ONNX Runtime, the CPU
pip install ./python -C cmake.define.CARBON_WITH_CUDA=ON    # and the CUDA kernels
```

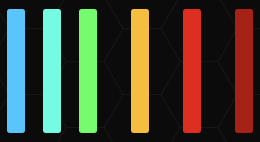
scikit-build-core builds the module `carbon_ocr` from this CMake project, with ONNX Runtime and without the CLI or the C++ tests. ONNX Runtime's library is installed next to the module, which finds it there. TensorRT is not in the module yet, so `Pipeline(device="cuda")` needs a build of `carbon-ocr` instead (`carbon-ocr ocr --device cuda`). python/README.md lists the module's API.

18 Tests

`ctest --preset <preset>` runs the C++ tests: the tensors, the crop rule, the CPU kernels against their scalar reference, the native backend against ONNX Runtime, the detector's and order model's decoders, the detector's two passes merged, the engine and the batched pipeline on small fake models, the HTTP parser and the server, PDFs, the output formats and the command line. The tests that need a GPU are built with CUDA or TensorRT and run on a machine with one.

```
pip install ./python pytest==8.4.2 pillow==13.0.0
python -m pytest tests/python/test_carbon_ocr.py
```

The Python tests hold the module to the C++, and its crop rule to Pillow's bilinear resize, byte for byte. `tests/python/test_ocrmypdf_plugin.py` runs the OCRmyPDF plugin on a built `carbon-ocr` (`CARBON_OCR` names it). It needs the packages in `tools/ocrmypdf/requirements.txt`, and `tesseract` on `PATH` for whole OCRmyPDF runs.



PART IV

Using

The command line and its HTTP server, the output formats, the Python module and the OCRmyPDF plugin.

19 Reading pages

20 Serving pages over HTTP

21 Output formats

22 Timing and inspecting

23 Python

24 OCRmyPDF

19 Reading pages

```
carbon-ocr ocr page.png scan.jpg --set models/fullerene-3M          # plain text
carbon-ocr ocr page.png --set models/fullerene-3M --backend ort     # all on ONNX Runtime
carbon-ocr ocr report.pdf --set models/fullerene-3M --dpi 300       # each page of a PDF
carbon-ocr ocr page.png --set models/fullerene-3M --format json     # boxes, words, timings
carbon-ocr ocr scan.pdf --set models/fullerene-3M --format pdf --output searchable.pdf
carbon-ocr ocr scans/*.png --set models/fullerene-3M --device cuda --engine-cache .engines
carbon-ocr ocr scans/*.png --set models/fullerene-3M --device cuda --precision fp32
```

`ocr` reads PNG, JPEG, BMP and PNM pages, and PDFs. A PDF's pages are rendered by PDFium in `carbon-pdf` helper processes. If a document crashes PDFium, only a helper dies, and your program keeps running.

A born-digital PDF page whose text layer passes the engine's checks is read from that layer instead of by OCR. Its pictures (the page's image objects) are still read by OCR, and their lines join the layer's. The JSON output gives each page's source (`ocr`, `text layer` or `hybrid`). For a PDF page read by OCR, it also says why the page's text layer was not used.

By default the pages run in parallel on the CPU, one worker per physical core. `--device cuda` reads them through the batched pipeline on the GPU instead: TensorRT runs the set's files at `--precision` (the FP16 files by default), and each model is batched across the pages in flight. This needs a build with TensorRT and CUDA (BUILD.md).

OPTION	DEFAULT	WHAT
`--set DIR`	required	the model set
`--rec NAME`	`rec-latin`	the recognizer: `DIR/NAME.onnx` and `DIR/NAME.vocab.json`
`--format F`	`text`	`text`, `json`, `hocr`, `alto`, `page`, `markdown`, `html` or `pdf`
`--output FILE`	standard output	the file to write; `pdf` needs one
`--out-dir DIR`		`page` (PAGE XML) writes one file per page here
`--text-out FILE`		also write the pages' text to this file, beside any format
`--text-only`	off	`pdf`: only the invisible text, without the page images
`--image-dpi N`	300	`pdf`: the dpi an image page is placed at
`--dpi N`	200	the dpi a PDF's pages are rendered at
`--text-layer auto|off`	`auto`	read a PDF page from its own text layer where it passes the checks
`--hybrid auto|off`	`auto`	read the pictures of a page that was read from its text layer by OCR

OPTION	DEFAULT	WHAT
`--second-pass auto|off`	`auto`	detect a page of tiny text (its median line under 9 px at the detector's scale) again at twice the scale, and merge the two passes
`--as-given`	off	no quarter turns and no deskew: the page as given
`--precision auto|fp32|fp16|int8`	`auto`	the files the models run. On the CPU: `fp16` the set's `.fp16.onnx` on the engine's own kernels, `int8` its `.int8.onnx`, `fp32` its `.onnx`; `auto` as below. On the GPU: `fp16` the `.fp16.onnx` files and the recognizer's trunk and fused CTC head, `fp32` the `.onnx` files, `int8` the `.int8-sym.onnx` files; `auto` the FP16 files, the FP32 one where a set lacks one
`--backend ort|native`	per model	run every model on one backend; by default a model runs on the native int8 backend where the set has its `.int8.onnx`, else on ONNX Runtime
`--workers N`	one per physical core	CPU workers
`--no-pin`	pinned	do not pin workers to cores
`--device cpu|cuda`	`cpu`	`cuda`: the batched pipeline on the GPU
`--batch N`	8	`cuda`: pages in a detection batch
`--in-flight N`	32	`cuda`: pages in the pipeline at once
`--engine-cache DIR`		keep TensorRT's built engines here, for later runs

The reading order comes from the set's `order.onnx` when the set has one; otherwise the engine orders lines by the XY-cut. The order model is also used only where it is sure of its result, and the XY-cut takes over elsewhere. The JSON output records which of the two ordered each page.

20 Serving pages over HTTP

```
carbon-ocr serve --set models/fullerene-3M                              # http://127.0.0.1:8080
carbon-ocr serve --set models/fullerene-3M --host 0.0.0.0 --port 9000   # every interface
carbon-ocr serve --set models/fullerene-3M --device cuda --engine-cache .engines
curl --data-binary @page.png http://127.0.0.1:8080/ocr                  # the page as JSON
curl --data-binary @page.png "http://127.0.0.1:8080/ocr?format=text"
curl --data-binary @scan.jpg "http://127.0.0.1:8080/ocr?format=pdf" -o searchable.pdf
curl http://127.0.0.1:8080/info
```

`serve` loads a set's models once and reads the pages clients send over HTTP/1.1, until Ctrl+C. Every client's pages share the engine. On the CPU each page goes to the next free worker, is decoded there and is answered as soon as it is read, so a slow page holds back no other. On the GPU (`--device cuda`) the batched pipeline puts every client's pages in its batches. `serve` takes `ocr`'s model, worker and device options (`--rec`, `--backend`, `--precision`,

`--second-pass`, `--as-given`, `--workers`, `--no-pin`, `--device`, `--batch`, `--in-flight`, `--engine-cache`), so it runs every model file the engine does. Connections are kept alive, and a client may send `Expect: 100-continue`.

REQUEST	ANSWER
`POST /ocr`	the body is a page's image (PNG, JPEG, BMP or PNM), and the answer that page as `ocr` writes it, in `?format=`: `json` (the default: the page's object), `text`, `hocr`, `alto`, `page`, `markdown`, `html` or `pdf`. `?name=` names the image in hOCR, ALTO and PAGE XML (`page` by default), and `?text_only=1` makes the `pdf` its invisible text alone. `?html=1` adds `html` to the `json` object: the page's body as `html` writes it. The headers `X-Carbon-Ms` and `X-Carbon-Lines` give the page's time in the engine and its number of lines
`GET /health`	`ok`
`GET /info`	JSON: the version, the set, the device, the precision, the workers, the pages in flight and the queue

STATUS	WHEN
400	a request it cannot read, or an unknown format
404, 405	an unknown path, or a method the path does not take
411	a body without a `Content-Length` (chunked bodies are not taken)
413	a body past `--max-body-mb`
415	a body that is no image it reads; a PDF among them, since PDFs go through `ocr`
417, 505	an `Expect` other than `100-continue`; an HTTP version other than 1.0 and 1.1
422	an image that does not decode
431	a request's head past 16 KB
500	the engine's error for that page; the pages after it are read as ever
503	the queue full, with `Retry-After: 1`

An error's body is its reason, as text.

OPTION	DEFAULT	WHAT
`--host ADDR`	`127.0.0.1`	the address to listen on: this machine only; `0.0.0.0` for every interface
`--port N`	8080	the port; 0 picks a free one, which the first line printed gives
`--queue N`	64	requests that wait past the pages in flight; more are answered 503
`--max-body-mb N`	64	the largest page a request may send
`--decoders N`	4, at most the cores	`--device cuda`: the threads that decode the pages
`--image-dpi N`	300	`format=pdf`: the dpi the page's image is placed at
`--quiet`	off	no line on standard error for each request

The server has no TLS and no authentication. To serve beyond the machine, put it behind a proxy that has them.

21 Output formats

FORMAT	WHAT
`text`	each page's lines in reading order, one per line, with a form feed between pages
`json`	one object with every page: its size, rotation and source, each line's box, text, confidence and words, the pictures, the order model's blocks with their labels and table cells, and timings
`hocr`	hOCR 1.2 (XHTML), the format Tesseract writes and OCRmyPDF reads: blocks, lines (headings and captions marked) and words with their boxes and confidences
`alto`	ALTO XML 4.4: text blocks with their labels, lines, words with their confidences
`page`	PAGE XML (2019-07-15), one file per page in `--out-dir`: regions typed by their labels, lines, words
`markdown`	headings, paragraphs, lists, tables (as HTML, from their cells) and captions, from the order model's blocks; running headers and footers are left out
`html`	an HTML5 document, a `<section>` per page: the order model's blocks as headings, paragraphs (a word broken by a hyphen at a line's end joined again), lists, tables from their cells, figures with their captions, running headers and footers (kept), footnotes and formulas
`pdf`	a searchable PDF: each page's image with its words as invisible text above it; `--text-only` writes the text layer alone

Boxes are in the page image's pixels. For a PDF page they are in the pixels of the page as rendered at `--dpi`.

22 Timing and inspecting

```
carbon-ocr version
carbon-ocr cpus                                                  # instruction sets and cores
carbon-ocr cpus --gemm                                           # the int8 GEMM timed per ISA
carbon-ocr pages models/fullerene-3M --workers 1,2,4,all         # a set timed page by page
carbon-ocr pages models/fullerene-3M --in-flight 1               # one page at a time: latency
carbon-ocr info  model.onnx                                      # inputs, outputs, metadata
carbon-ocr bench model.onnx --backend ort --shape x=64x1x32x640 --runs 200
carbon-ocr bench model.int8.onnx --backend native --shape x=8x1x32x512
carbon-ocr bench model.onnx --backend trt --shape x=64x1x32x640 --engine-cache .engines
carbon-ocr bench model.onnx --workers 1,2,4,all                  # a pinned worker per core
carbon-ocr compare model.onnx --shape x=8x1x32x512               # native against ONNX Runtime
```

- **`cpus`** prints the instruction sets the kernels use and the cores the workers run on. `--gemm` times the int8 GEMM on one thread with each instruction set, at the shapes of the models' convolutions.
- **`pages`** times a set the way the CPU pipeline runs a page: orientation beside detection, the line decoder, the lines in tasks any idle worker can take, then the order model. Its inputs are fixed at the models' design sizes. `--in-flight 1` measures one page at a time on every core (latency). By default every page is queued at once (throughput). `--det auto|one|wide|strips` picks how detection uses the cores.
- **`bench`** times one model with fixed inputs and reports the median, p90 and fastest run. On TensorRT it times two runs: from host memory with the copies included, and on device buffers, the way the GPU pipeline runs a model.

`--opt-level 0-5` sets TensorRT's builder optimization level (default 3), and `--no-graphs` turns off CUDA graphs. `--isa NAME` holds the native backend to one instruction set. `--workers` runs a pool of pinned workers, each with its own session, and reports their scaling.

- **`compare`** runs a model on the native backend and on ONNX Runtime and reports how far apart their outputs are. `--against FILE` gives ONNX Runtime a different file, such as the FP32 model for an int8 one.

23 Python

```
pip install ./python
```

```
import numpy as np
import carbon_ocr

engine = carbon_ocr.Engine("models/fullerene-3M", workers=0)        # 0: one per core
pages = engine.ocr([carbon_ocr.load_image("a.png"), carbon_ocr.load_image("b.png")])
for page in pages:
    print(page["rotation"], page["order"])                          # "model" or "xy-cut"
    for line in page["lines"]:                                       # in reading order
        print(line["box"], round(line["confidence"], 3), line["text"])

for name in ["a.png", "b.png", "c.png"]:                            # a stream: each page starts
    engine.submit_file(name)                                         # when a worker is free
while (page := engine.next()) is not None:                           # in the order given
    print(page["ms"]["load"], page["text"][:60])

pipeline = carbon_ocr.Pipeline("models/fullerene-3M", page_batch=8)  # batched; same results
pages = pipeline.ocr([carbon_ocr.load_image("a.png")])
print(pipeline.stats)

session = carbon_ocr.Session("models/fullerene-3M/rec-latin.onnx", backend="ort")
logits, = session.run([np.zeros((1, 1, 32, 512), np.float32)])     # [1, 800, 128]
```

The module also exposes the engine's building blocks: the crop rule (`crop` , `crop_lines` , `crop_region`), the CTC decoders (`ctc_greedy` , `ctc_collapse`), the detector's decoder and the merge of its two passes (`decode_det` , `takes_second_pass` , `merge_passes`), the order model's features and decoder (`order_features` , `decode_order` , `reading_order`), and text scoring (`text_errors` , `char_confusions`). python/README.md lists all of them.

24 OCRmyPDF

`tools/ocrmypdf/carbon_ocrmypdf.py` plugs Carbon-OCR into OCRmyPDF 17 as its OCR engine. OCRmyPDF prepares each page itself, turning and deskewing it, and the plugin runs `carbon-ocr ocr --as-given` on the page. With OCRmyPDF's default renderer, the plugin

returns hOCR. With `--pdf-renderer sandwich`, it returns a text-only PDF at the image's own dpi.

```
pip install -r tools/ocrmypdf/requirements.txt
ocrmypdf --plugin tools/ocrmypdf/carbon_ocrmypdf.py \
    --carbon-set models/fullerene-3M in.pdf out.pdf
```

The plugin's options:

- `--carbon-set DIR`: the model set. Without it, the plugin uses the set named by `CARBON_OCR_SET`.
- `--carbon-ocr PATH`: the program. Without it, the plugin uses `CARBON_OCR`, then the `carbon-ocr` on `PATH`.
- `--carbon-workers N`: the number of workers.

OCRmyPDF checks for `tesseract` on every run, so Tesseract must be installed, though the plugin never runs it.

Carbon OCR

Every line read, every page in order.

Every number measured, in public.

EVERY BYTE THE SAME

The crop rule and the decoders are written once, so the CPU and the GPU give the same bytes, and so do one core and many.

EVERY CORE USED

A pinned worker per physical core, and work cut smaller than a page, so one page alone uses every core too.

EVERY PAGE IN FLIGHT

On the GPU, each model runs on batches drawn from all the pages in flight, and only ids and confidences come back.

Where it stands

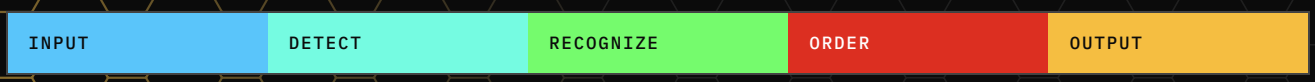
The engine is built and tested, on CPUs and NVIDIA GPUs.

The Fullerene weights are out, under OpenRAIL-AMS.

Karan Vora, Founder · Carbon Research

Carbon-OCR is the engine and **Fullerene** its models. This guide is built from `docs/ENGINE.md`, `docs/BUILD.md` and `docs/USAGE.md` in the repository.

Measured numbers come from random weights on a laptop (i7-13620H, RTX 4060 Laptop GPU), 27 to 30 October 2026: speed does not depend on the weights.



Karan Vora, Founder · Carbon Research

getcarbonresearch.com